MeshCutting 설명서

MiniEngine(DirectX 12) 위의 실시간 메시 절단 — NDC25「저 메시의 목을 쳐라!」 방식의 구현 구조와 알고리즘

기준 커밋 `223d2a1` (2026-10-22 13:17,「모델 업데이트」)에서 시작했다. 작성 중 `4f4bfb0`「절단 조각을 평면 법선 방향으로만 벌리기」(14:24, `CutDebugView.cpp`만 변경)가 추가되어, 영향받는 부분(4.9, 6.1, 부록 A #7)은 `4f4bfb0` 기준으로 고쳤다. 파이프라인 코드의 마지막 큰 변경은 `cb1c12a`「Unify procedural mesh cutting pipeline」이다.

대상 코드 `MiniEngine/MeshCutting/`, `MiniEngine/ModelViewer/ModelViewer.cpp`, `MiniEngine/Model/`의 관련 부분

작성 방법 문서(`Docs/`)를 읽은 뒤 코드를 직접 읽어 대조했다. 문서와 코드가 다르면 **코드를 기준**으로 썼고 차이는 부록 A에 모았다. 빌드·실행·화면 확인은 하지 않았다.

한 문단 요약

평면을 놓고 `Enter`를 누르면, 평면이 지나는 메시 범위를 원본 모델에서 **`ProceduralMesh` 하나**로 복사한다. 그 메시를 현재 자세로 옮긴 뒤 평면으로 잘라 양쪽 두 개의 `ProceduralMesh`를 얻는다. 원본에서는 복사한 범위를 숨긴다. 스켈레탈 메시는 버텍스 컬러 마스크로 숨기고, 스태틱 메시는 메시 전체를 숨긴다. 스켈레탈은 해당 joint를 디버그용 `BreakConstraint`로 끊는다. 이후 프레임마다 두 조각은 각자 맞는 행렬로 자세를 갱신하고 같은 객체가 그대로 그려진다. 복사·용접·자세 갱신·절단·표시가 모두 `ProceduralMesh`라는 하나의 타입을 거친다. 이것이 최근 통합(`cb1c12a`)의 핵심이다.

핵심 사실 7가지

1. **메시 타입은 하나:** import, 용접, 자세 갱신, 절단 결과, 렌더 캐시가 모두 `ProceduralMesh` 안에 있다. 예전의 `WeldInput`·`PatchPiece`는 없어졌다.
2. **위치는 기하 정점(`MeshVertex`)이 소유:** 렌더 정점(`MeshCorner`)은 정점 번호만 가리킨다. 그래서 UV seam 양쪽과 cap 경계의 위치가 비트 단위로 같다.
3. **스킨 정보는 공유 표:** 용접 후 원본 정점의 스킨은 `Skinning(const)` 표 하나에 있다. 잘린 두 조각이 이 표를 `shared_ptr`로 공유한다. 새 교점은 최대 3개 원본 정점의 무게중심 조합(`MeshVertexOrigin`)으로 표현한다.
4. **절단 시점은 Enter 한 번:** 편집 중에는 교차 삼각형으로 후보 joint/메시만 고른다(`CutPick`). 복사·용접·절단은 Enter 프레임에 작업마다 한 번 한다. Unreal의 "맞은 순간 복사→자르기"와 같은 시점이다.
5. **절단은 원자적:** `SliceGeometry`는 실패하면 결과를 비운다. `SliceProceduralMesh`는 두 결과가 모두 만들어진 뒤에만 입력 메시를 바꾼다. 유한 사각형 밖으로 루프가 나가면 컨트롤러가 그 작업을 확정하지 않는다(`LoopOutsidePlane`).
6. **cap은 직접 구현한 ear clipping:** 루프 정점만 써서 새 점 없이 삼각화한다. 구멍은 브리지로 잇는다. 그래서 표면과 cap 사이에 틈이 생기지 않는다.
7. **물리는 없다:** 스켈레탈 분리는 끊긴 노드를 world에 고정하고 법선 방향으로 조금 옮기는 디버그 방식이다. 충돌·중력·래그돌은 구현되어 있지 않다.

목차

1. 큰 그림 — 모듈과 의존 방향
2. 데이터 구조 — ProceduralMesh와 주변 타입
   2.4 Unreal 대응 · 2.5 통합 전후 비교
3. 원본 → ProceduralMesh 복사 흐름
4. 절단 알고리즘
   분류 · 분할 · 루프 · cap · 원자성 · 스키닝 · 숨김 · 분리
5. 전체 실행 흐름과 입력 키
6. 현재 한계와 하지 않은 것
7. 파일별 역할 표
8. 용어집
부록 A. 문서–코드 차이
부록 B. 확인하지 못한 것

1. 큰 그림 — 모듈과 의존 방향

MeshCutting은 세 층으로 나뉜다. **Geometry**는 순수 계산이고 Model·Core·D3D12에 의존하지 않는다. 그래서 `tests/PlaneSlicerTests.cpp`가 GPU 없이 빌드된다. **원본 읽기와 조율**(BonePatch, StaticMeshCopy, Runtime)은 MiniEngine의 `Model`/`ModelInstance`를 읽는다. **표시**(Rendering, Debug)는 D3D12로 그린다. `ProceduralMesh`는 그 가운데에서 모든 층이 주고받는 공통 객체다.

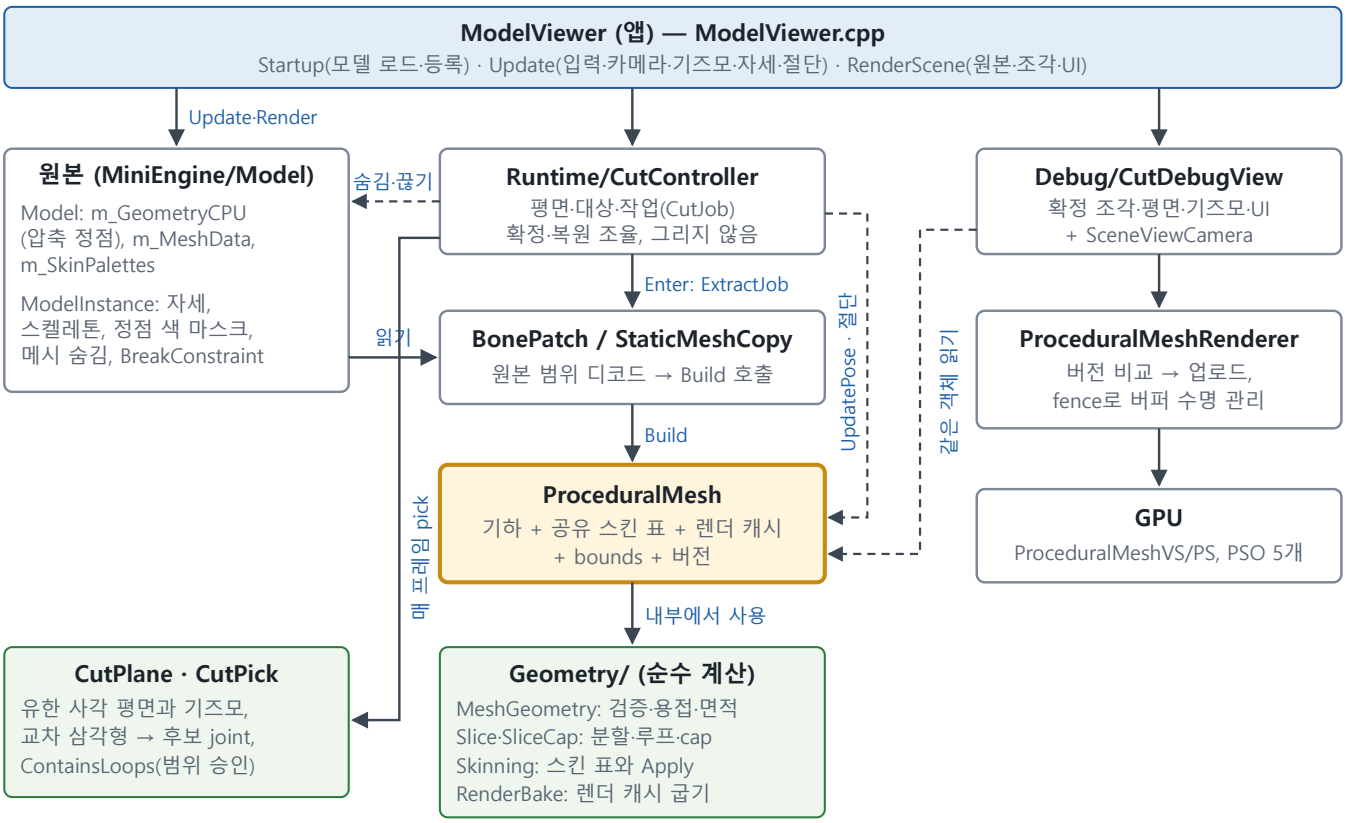


그림 1. 모듈 구성과 호출 방향. 녹색은 Model·Core에 의존하지 않는 순수 계산, 노란색이 공통 메시 객체다. 점선은 상태 변경이나 공유 참조다.

1.1 한 번의 절단을 객체 하나로 따라가기

단계	호출 경로 (실제 함수 이름)	무엇이 일어나나
등록	`ModelViewer::Startup` → `CutController::RegisterInstance` (캐릭터 1 + 정적 모델 2)	대상의 스킨 팔레트와 스킨 없는 메시 목록(`ReloadPalettes`)을 기록한다. 캐시는 아직 만들지 않는다.
후보 선택	`Update` → `UpdateCandidates` → `RefreshPick` → `CutPick::PickJoints` / `PickStaticMeshes` → `DiffJobs`·`ApplyDiff`	평면과 실제로 교차하는 삼각형이 joint(또는 스태틱 메시)를 고른다. 그 결과로 작업 key 목록만 갱신한다.
생성 (Enter)	`ConfirmAll` → `ConfirmJob` → `ExtractJob` → `ExtractBonePatch` | `CopyProceduralMeshFromStaticMesh` → `ProceduralMesh::Build`	원본 범위를 디코드하고 용접해 `ProceduralMesh`를 만든다. 원본 출처는 `MeshSource`에 담긴다.
현재 자세	`SkinPatch` → `BuildJobMatrices` → `ProceduralMesh::UpdatePose`	스킨 행렬 또는 강체 world 행렬로 기하·렌더 위치·bounds를 함께 갱신한다.
절단	`SliceWithPlane` → `SliceProceduralMesh` → `SliceGeometry` → `CutPlane::ContainsLoops`	무한 평면으로 자른 뒤, 루프가 모두 유한 사각형 안인지 확인한다.

확정	`ConfirmJob`: `SetOriginalVisible(false)`·`SetBreakOf` / `ConfirmAll`: `UpdateSkinMask`·`UpdateBrokenJoints`	스태틱 원본을 숨긴다. 스켈레탈은 마스크와 BreakConstraint를 걸고, 절단 순간의 행렬을 저장한다.
후속 프레임	`SkinJobs` → `BuildPieceMatrices` → `UpdatePose` (조각마다)	남는 쪽과 떨어지는 쪽이 각자 맞는 joint 행렬로 움직인다.
표시	`RenderScene` → `CutDebugView::Draw` → `ProceduralMeshRenderer::Draw` → `NotifySubmitted`	확정 메시 객체를 그대로 그린다. 바뀐 버전만 업로드한다.

2. 데이터 구조 — ProceduralMesh와 주변 타입

`ProceduralMesh`(`ProceduralMesh.h`)는 절단에 쓰는 기하와 GPU에 보낼 렌더 캐시를 함께 소유한다. 기하는 공개 API로만 바뀌고, 읽기 접근자(`GetGeometry`, `GetVertices`, `GetCorners`, `GetFaces`, `GetData`, `GetBounds`)는 모두 `const`다.

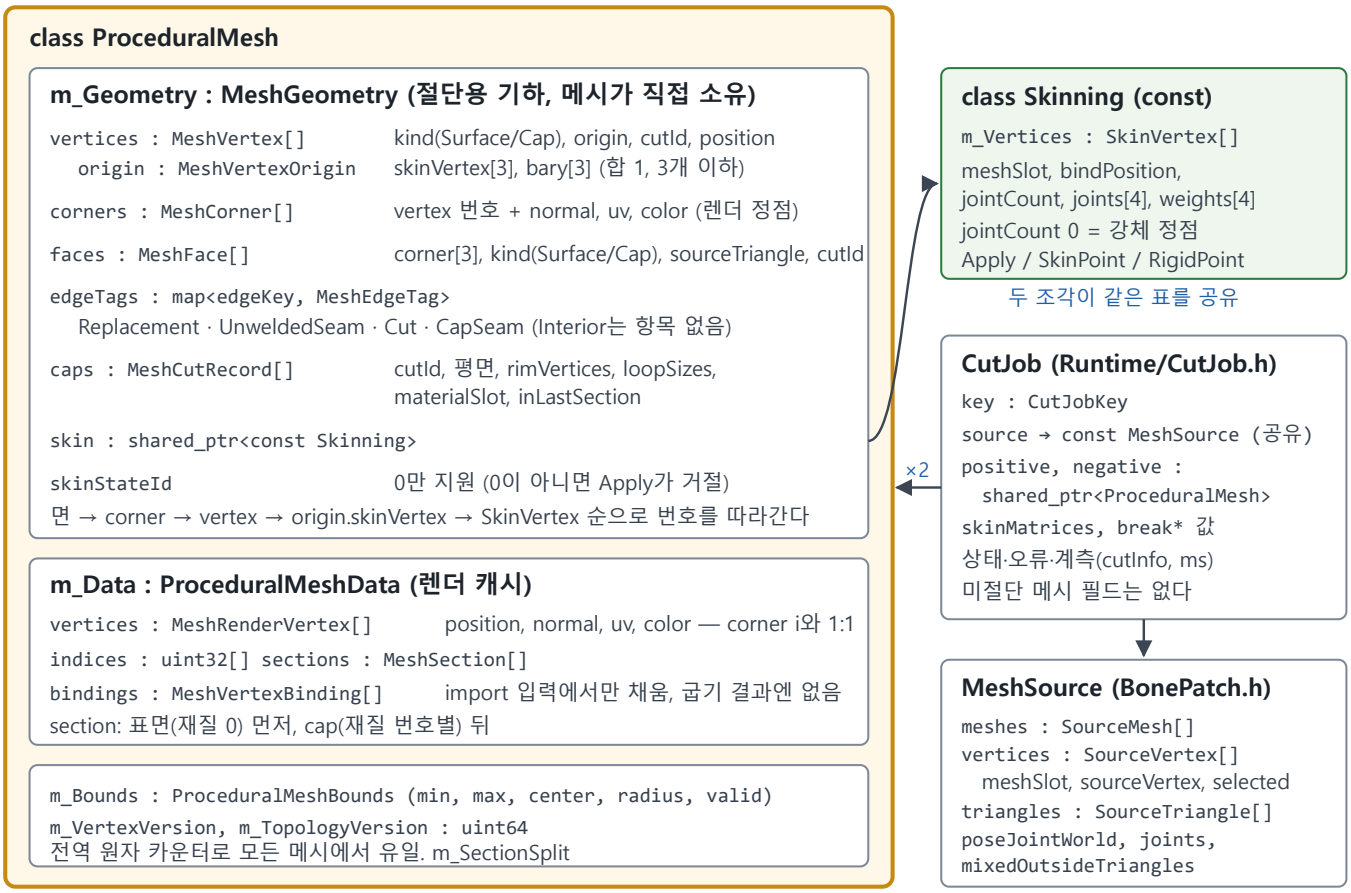


그림 2. ProceduralMesh가 소유하는 것과 주변 타입. 기하 배열은 조각마다 따로 갖고, Skinning 표만 공유한다.

2.1 소유하는 것

멤버	내용과 규칙
`m_Geometry` (`MeshGeometry`)	절단 계산의 기준 데이터다. **위치는 `MeshVertex::position`에만 있고**, 렌더 정점(`MeshCorner`)은 그 번호만 가리킨다. `MeshVertexOrigin`은 Skinning 표의 원본 정점 최대 3개와 무게중심 가중치다. 처음 만든 정점은 자기 자신 하나(가중치 1)이고, 교점은 양 끝의 조합이다. `MeshFace::cutId`는 Cap 면이면 그 면을 만든 절단, Surface 면이면 그 면을 나눈 절단(0이면 나뉜 적 없음)이다.
`edgeTags`	정렬된 (vertexA, vertexB) 64비트 key → 종류. 열린 경계는 `Replacement`(복사 범위의 경계), `UnweldedSeam`(용접하지 않은 seam), `Cut`(cap 없는 절단 경계)이다. `CapSeam`은 cap과 표면이 공유하는 내부 에지다. 태그 없는 열린 에지는 검증에서 거절된다.

`caps` (`MeshCutRecord`)	절단 하나에 기록 하나씩, 양쪽 조각에 각각 붙는다. 평면 점·바깥 법선·rim 정점(모든 루프, 외곽 먼저)·루프별 크기를 담는다. cap 재질 규칙(`materialSlot`, `inLastSection`)도 여기에 있다.
공유 Skinning 표	`shared_ptr<const Skinning>`. 용접 후 원본 정점마다 `SkinVertex` 하나를 둔다(joint 오름차순, 양의 가중치만). 잘린 조각들은 기하 배열을 각자 갖고 이 표만 공유한다.
렌더 캐시 `m_Data`	`BuildRenderMesh`(`Rendering/RenderBake.cpp`)가 기하에서 굽는다. **렌더 정점 i = corner i**이고 위치는 그 corner가 가리키는 기하 정점에서 복사한다. section은 표면(재질 0)을 먼저 두고(디버그 분할 시 [나뉜 적 없는 면][나뉜 면]), 이어서 cap 재질 번호마다 하나씩 둔다.
`m_Bounds`	렌더 정점에서 계산한 AABB와 중심 기준 반지름. 검증(`ValidateVertices`)과 같은 루프에서 만든다.
버전 2개	`m_VertexVersion`은 위치가 바뀌면, `m_TopologyVersion`은 위상(인덱스)이 바뀌면 새 값을 받는다. 값은 전역 원자 카운터 `NextVersion()`에서 나오므로, 같은 주소를 재사용한 다른 메시와 우연히 겹치지 않는다. 렌더러는 이 값만 비교해 업로드 여부를 정한다.

2.2 공개 API

함수	하는 일
`SliceStatus Build(const ProceduralMeshData&, WeldStats&, string*)`	원본 import 결과(정점 + `bindings`)를 `BuildMeshGeometry`로 한 번 용접한다. 그 결과를 `SetTopology`에 넘긴다. 복사 함수 두 개가 모두 이 함수를 부른다.
`bool SetTopology(MeshGeometry, string*)`	기하를 검증하고(`ValidateMeshGeometry`), 렌더 캐시를 굽고, bounds와 두 버전을 갱신한 뒤 기하를 소유한다. 실패하면 기존 상태가 그대로 남는다. 절단 결과도 이 함수로 들어온다.
`bool UpdatePose(const vector<vector<XMFLOAT4X4>>& matrices, string*)`	슬롯별 행렬로 기하 정점 위치를 다시 계산한다(`Skinning::Apply`). 그 위치를 렌더 정점에 복사하고 bounds를 갱신한다. 위상은 바꾸지 않으며, 모든 검사가 끝난 뒤 한꺼번에 반영한다. **법선은 갱신하지 않는다.**
`bool SetSectionSplit(bool splitByOrigin, string*)`	디버그용이다. 표면을 [원래 면][절단으로 나뉜 면] section으로 나눠 다시 굽는다. 기하는 그대로이고 렌더 캐시와 버전만 바뀐다.
`bool SetMesh(ProceduralMeshData, string*)` `bool UpdateVertices(...)`	기하 없는 **렌더 전용** 메시(디버그 튜브, 평면, 기즈모)에 쓴다. 절단 기하가 있는 메시에 `UpdateVertices`를 부르면 거절한다.
`void Clear()`	모두 비우고 버전을 새로 받는다.

절단 진입점은 멤버가 아니다. `Geometry/Slice.h`의 자유 함수 `SliceProceduralMesh(ProceduralMesh& inOutMesh, planePosition, planeNormal, createOtherHalf, ProceduralMesh* outOtherHalf, SliceCapOption, capMaterialSlot, SliceStatus*, string*, vector<vector<uint32_t>>* outLoops)`다. Unreal의 `UKismetProceduralMeshLibrary::SliceProceduralMesh`와 인자 순서를 맞췄다.

2.3 주변 타입

타입 (파일)	보관하는 것
`MeshSource` (BonePatch.h)	원본 추적용이다. 기여한 메시(`SourceMesh`: meshIndex, world 행렬 번호, 팔레트 시작), import 정점 순서의 `SourceVertex`(슬롯, 원본 정점 번호, `selected`), 삼각형 출처(`SourceTriangle`), 추출 시점 joint world(`poseJointWorld`), `mixedOutsideTriangles`를 담는다. 위치·법선·스킨 사본은 두지 않는다.
`CutJob` (Runtime/CutJob.h)	작업 key(대상, 종류, 팔레트/메시 번호, joint), 출처, 확정 메시 두 개(`shared_ptr`), 분리 스냅샷(`breakCutMatrices`, `breakNodeWorld`, `breakParentWorld`, `breakDetached`, `positiveFalls`), 상태·오류·계측을 담는다.

`CutTargetSource`, `CutStaticMesh`	후보 선택(pick) 전용 캐시다. 팔레트 전체 joint로 읽은 정점·스킨·인덱스(`CutPickMesh`), 스태틱 메시의 로컬 위치·인덱스를 담는다. 절단 메시와는 목적이 다르다.
`CutPlane` (Geometry/CutPlane.h)	world의 유한 사각형(`CutPlaneShape`: 중심, 쿼터니언, 반 크기)과 기즈모 상태를 담는다. 슬라이서에는 `ToSlicePlane()`으로 무한 평면을 넘기고, 사각형은 `ContainsLoops`에서만 쓴다.

2.4 Unreal ProceduralMeshComponent와의 대응

Unreal	이 프로젝트	차이
`UProceduralMeshComponent` (섹션별 정점·인덱스)	`ProceduralMesh` (렌더 캐시 `m_Data` + 절단 기하)	Unreal은 렌더 정점만 갖는다. 여기서는 용접된 기하·스킨 출처·경계 태그를 함께 소유한다. 컴포넌트 transform 대신 정점이 world 공간에 있다.
`CopyProceduralMeshFromStaticMeshComponent`	`CopyProceduralMeshFromStaticMesh` (StaticMeshCopy.h)	컴포넌트 전체가 아니라 메시 하나 단위로 복사한다. 재질 section은 복사하지 않는다 (section 하나).
(NDC 발표자의 스켈레탈 범위 복사)	`ExtractBonePatch` (BonePatch.h)	Unreal 공개 API에는 없다. NDC 슬라이드의 "본 영향 범위를 프로시저럴 메시로 복사"에 해당한다.
`SliceProceduralMesh(InProcMesh, PlanePosition, PlaneNormal, bCreateOtherHalf, OutOtherHalfProcMesh, CapOption, CapMaterial)`	`SliceProceduralMesh(inOutMesh, planePosition, planeNormal, createOtherHalf, outOtherHalf, capOption, capMaterialSlot, ...)`	재질 대신 재질 번호를 받는다. `bool`을 반환하고 상태·오류·루프를 추가로 내보낸다. 평면을 로컬로 옮기지 않는다. 같은 재질 번호의 cap은 한 section에 모은다.
`EProcMeshSliceCapOption`	`SliceCapOption`: NoCap / CreateNewSectionForCap / UseLastSectionForCap	NoCap이어도 루프 추출·검증은 한다.
섹션 bounds로 조기 판정 추론	`CutPick::SphereMayCross` → `CountCrossedTriangles`	Unreal 내부는 소스로 확인하지 않았다(NDC25Reference 7절).
`SetVisibility` / 버텍스 컬러 마스킹	`ModelInstance::SetMeshVisibility` / `SetVertexColorOverride`	마스크는 정점 색 R ≥ 0.5일 때 color·depth PS에서 `discard`한다.
`BreakConstraint(BoneName)`	`ModelInstance::BreakConstraint(offset, nodeIndex)`	물리 관절을 끊는 것이 아니다. 노드 world를 고정하고 이동량을 더하는 디버그 대체물이다.
(ProceduralMesh는 스키닝 미지원 → NDC는 매 틱 정점 갱신)	`ProceduralMesh::UpdatePose` (CPU 스키닝)	NDC와 같은 CPU 갱신 방식이다.

2.5 통합 전과 비교 (`cb1c12a`에서 바뀐 것)

통합 전 이름은 `cb1c12a^`의 헤더(`BonePatch.h`의 `BonePatchSource`·`MakeWeldInput`, `Geometry/Patch.h`의 `PatchPiece`·`WeldInput`·`BuildPatchPiece`)로 확인했다. 아래 그림의 이전 흐름은 그 이름과 `ProceduralMeshSimplification.md`를 바탕으로 그렸다.

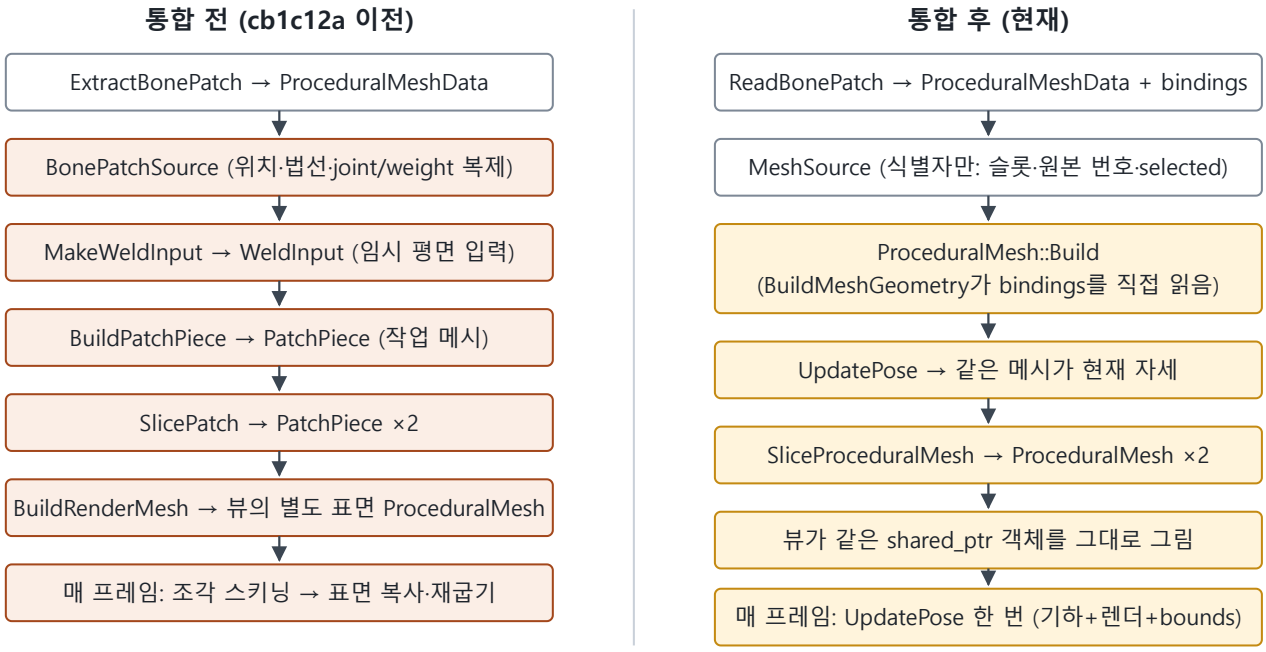


그림 3. 통합 전후의 메시 흐름. 주황은 없어진 중간 타입·복사, 노랑은 공통 ProceduralMesh 경로다.

없앤 것	대신 쓰는 것
`WeldInput`, `WeldInputVertex`, `MakeWeldInput`	`ProceduralMeshData::bindings`(`MeshVertexBinding`)를 `BuildMeshGeometry`가 직접 읽는다.
`PatchPiece` (작업 메시 타입)	`MeshGeometry`를 `ProceduralMesh`가 직접 소유한다.
`PatchNode/Corner/Face/Origin/Cap`	`MeshVertex/Corner/Face/VertexOrigin/CutRecord`로 이름을 통일했다.
`Geometry/Patch.h/.cpp`	`Geometry/MeshGeometry.h/.cpp`. 자료구조 정의는 `ProceduralMesh.h`로 옮겼고 계산만 남겼다.
`BonePatchSource`의 위치·법선·스킨 복제	스태틱에도 쓰는 `MeshSource`로 바꿨다. `SourceVertex`에는 식별자와 `selected`만 남았다.
`CutJob`의 미절단 patch, `CutJobSource` 래퍼	Enter 때 지역 `shared_ptr<ProceduralMesh>`로 만들어 자르고, 성공한 두 메시만 저장한다.
CutDebugView의 별도 표면 메시와 프레임별 재굽기	뷰는 확정 메시와 같은 `shared_ptr`를 보관하고 그대로 그린다. 경계 튜브만 따로 굽는다.

남긴 것: 원본 import 형식(`ProceduralMeshData`)은 한 번의 입력과 렌더 캐시 형식으로 쓰인다. 교점 생성, 실패 원자성을 위한 임시 배열, GPU용 정점 배열도 그대로 있다. 통합은 이런 내부 계산 배열까지 없앤 것이 아니다.

3. 원본 → ProceduralMesh 복사 흐름

MiniEngine은 모델을 읽은 뒤 정점을 압축된 바이트로만 보관한다(`Model::m_GeometryCPU`). `ModelLoader.cpp`는 메시 절단이 읽을 수 있도록 이 CPU 사본을 모든 모델에서 지우지 않는다(Unreal `bAllowCPUAccess`에 해당). 두 복사 경로는 이 바이트를 직접 디코드한다.

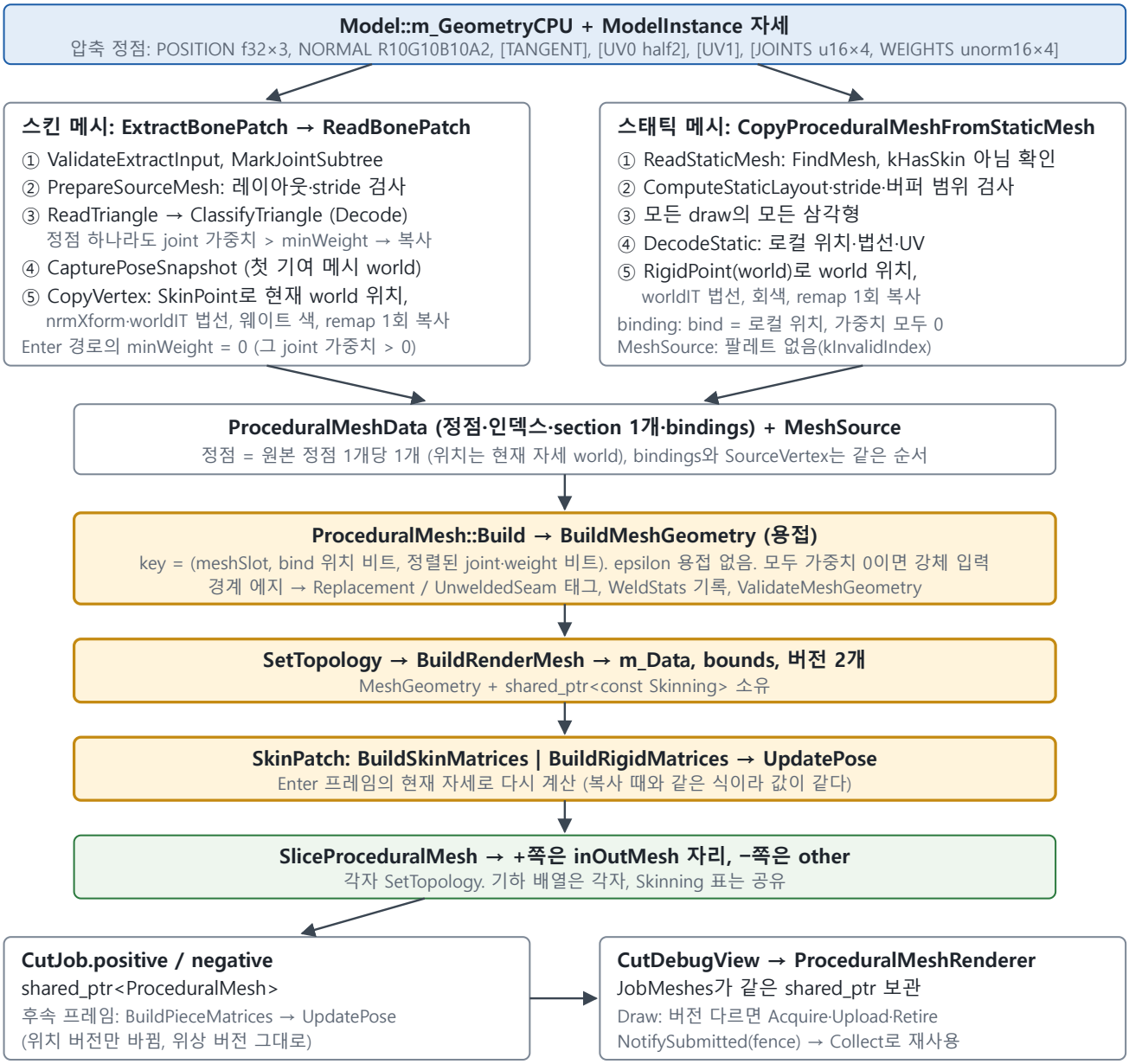


그림 4. 두 복사 경로가 하나의 ProceduralMesh로 모인 뒤, 절단·저장·업로드까지 가는 데이터 흐름.

3.1 압축 정점 디코드

속성	저장 형식	복원 (`Decode` / `DecodeStatic`)
위치	R32G32B32_FLOAT, 오프셋 0	그대로 `memcpy`. 바인드 자세(스킨) 또는 메시 로컬(스태틱) 좌표.
법선	R10G10B10A2_UNORM, 오프셋 12	10비트씩 `v/1023·2−1`.
탄젠트·UV1	있으면 4바이트씩	건너뛰기만 한다(오프셋 계산에만 쓴다).
UV0	R16G16_FLOAT	`XMConvertHalfToFloat`. 없으면 (0,0).
joints / weights (스킨만)	R16G16B16A16_UINT / R16G16B16A16_UNORM	가중치를 `/65535`한 뒤 합이 1이 되게 다시 정규화한다.

레이아웃은 `psoFlags`(`kHasTangent`, `kHasUV0`, `kHasUV1`)로 계산한다. 계산한 stride가 메시의 `vbStride`와 다르면 실패한다. 인덱스는 16/32비트 모두 읽고, `baseVertex`를 더한다. 같은 원본 정점은 `remap`으로 한 번만 복사한다.

3.2 스킨 경로의 선택 규칙과 자세

- **무엇을 복사하나**: `ClassifyTriangle`이 세 정점 중 하나라도 `Skinning::InJointSet`(그 joint의 가중치 합 > `minWeight`)이면 삼각형을 복사한다. Enter 때 `ExtractJob`은 `minWeight = 0.0`을 넘긴다. 후보 선택(`kPickMinWeight = 0.05`)과 기준이 다르다.
- **경계 정점**: 복사 범위 경계의 정점은 그 joint 가중치가 0이다. 그래서 원본과 패치의 이음매가 벌어지지 않는다는 것이 NDC 해석의 전제다(NDC25Reference 5절, `추론`).
- **mixedOutsideTriangles**: 복사하지 않은 삼각형 중 선택 joint의 자손과 바깥 joint에 함께 가중치가 있는 것을 센다. 분리하면 늘어날 수 있는 삼각형이다.
- **자세**: 위치는 `Skinning::SkinPoint(slotMatrices, joints, weights, bind)`다. 행렬은 `MakeSlotMatrices`: `meshWorld × joint.posXform`이다. 이후 매 프레임 스키닝도 같은 함수를 쓰므로, 복사 직후 위치와 첫 갱신 위치가 같은 식에서 나온다. 스태틱은 `RigidPoint(world, local)`를 쓰며, `Skinning::Apply`의 강체 정점과 비트 단위로 같다.
- **한 패치 = 한 world**: 패치에 기여하는 메시들의 world 행렬이 서로 다르면 `CapturePoseSnapshot`이 실패한다.

3.3 용접 (`BuildMeshGeometry`, `Geometry/MeshGeometry.cpp`)

용접 key = (meshSlot, 바인드 위치 3성분의 비트, 양의 가중치만 남겨 joint 오름차순으로 정렬한 (joint, weight 비트) 쌍). `-0`과 `+0`은 같은 값으로 본다.
• 위치가 같아도 스킨이 다르면 합치지 않는다. epsilon 용접은 없다(맞닿은 별도 shell이 합쳐지지 않게).
• 첫 렌더 정점의 위치를 기하 정점이 갖고, 같은 key의 다른 렌더 정점은 같은 번호를 가리키는 corner가 된다.
• 기하 정점 n의 `origin = {skinVertex[0]=n, bary[0]=1}`이다. Skinning 표의 n번 `SkinVertex`가 그 스킨이다.
• 경계 에지(면 하나만 쓰는 에지): 같은 슬롯의 바인드 위치 쌍, 또는 다른 슬롯의 world 위치 쌍을 덮는 경계 에지가 둘 이상이면 `UnweldedSeam`이다. 아니면 `Replacement`다.
• 가중치가 모두 0인 정점만 있으면 강체 입력으로 받는다. 일부만 0이면 `InvalidInput`이다.

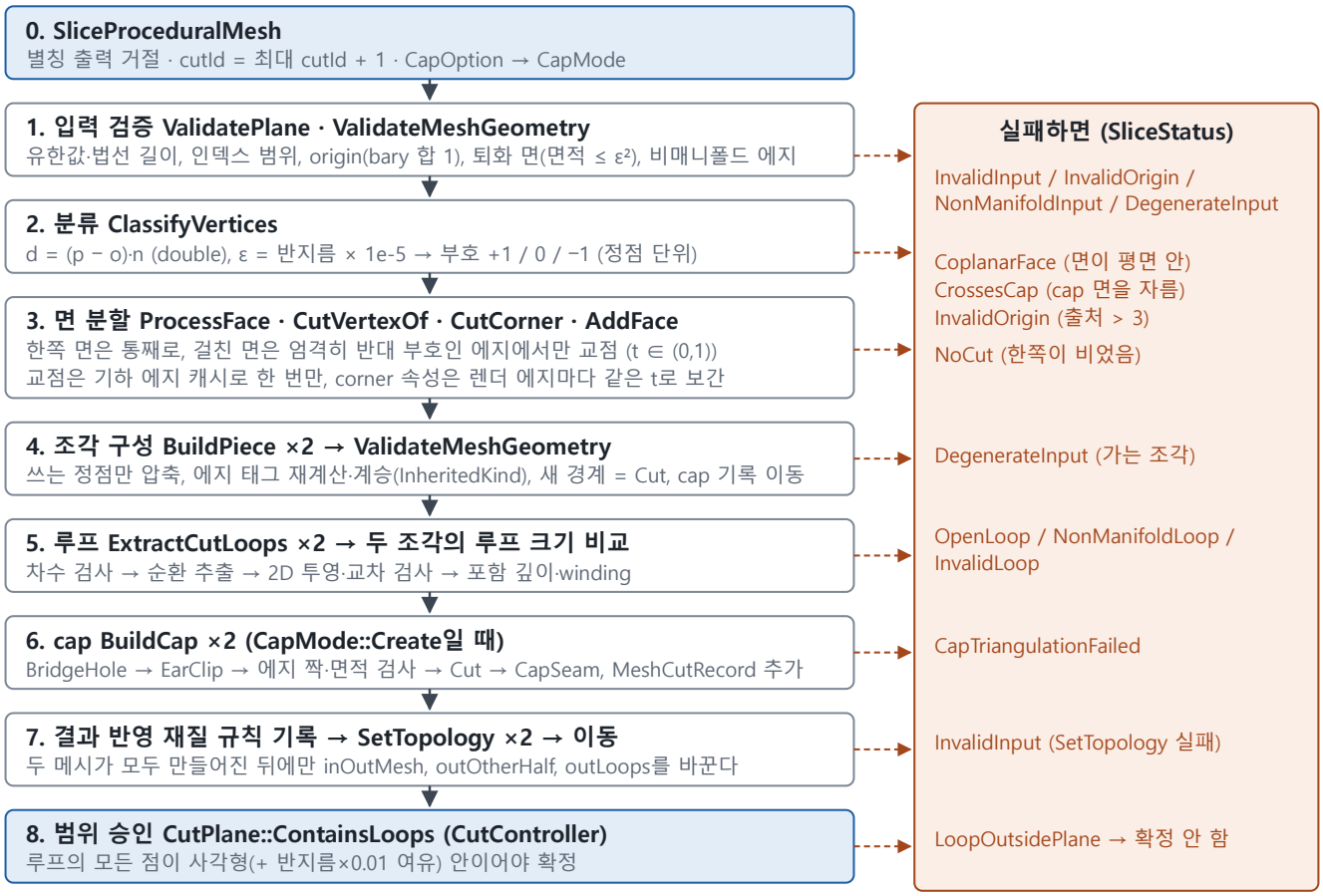
3.4 렌더러 업로드 (버전·fence)

1. `ProceduralMeshRenderer::Draw`는 메시 주소를 key로 `MeshGpu`(정점·인덱스 버퍼, 마지막으로 올린 두 버전)를 찾는다.
2. 위치 버전이 다르면 정점 버퍼만, 위상 버전이 다르면 인덱스 버퍼까지 새로 받는다(`Acquire` → `Upload`). 예전 버퍼는 `Retire`로 보낸다. 매 프레임 `UpdatePose`는 위치 버전만 바꾸므로 정점 버퍼만 다시 올린다.
3. 그린 버퍼는 `m_Pending`에 들어간다. `NotifySubmitted(fence)`가 그 fence를 기록하고, `Collect`가 GPU가 끝낸 버퍼를 재사용 목록으로 옮긴다.
4. 뷰는 작업이 사라지면 `ReleaseMesh`를 먼저 부르고 그 뒤에 메시를 놓는다. 확정 메시는 `shared_ptr`라서, 컨트롤러가 놓아도 뷰가 해제할 때까지 주소가 살아 있다.

PSO는 5개다: 양면, 후면 컬링, 와이어(양면/컬링), 깊이 없는 오버레이(기즈모). VS는 `World × ViewProj`만 곱하고 색을 넘긴다. PS는 정점 색 또는 section별 고정 색(디버그 tint)을 낸다. **조명은 없다.**

4. 절단 알고리즘

공개 진입점은 `SliceProceduralMesh`, 실제 계산은 `SliceGeometry`(내부 클래스 `Slicer`, `Geometry/Slice.cpp`)다. 루프와 cap은 `Geometry/SliceCap.cpp`가 맡는다. 유한 사각형 범위 승인은 컨트롤러(`CutController::SliceWithPlane`)가 한다.



어느 단계에서 실패하든 입력 메시는 그대로다. SliceGeometry는 결과를 비우고 status·detail만 남긴다.
LoopOutsidePlane일 때는 지역 메시만 잘린 채 버려진다. 작업은 확정되지 않고 원본 숨김도 일어나지 않는다.

그림 5. 절단 파이프라인과 단계별 실패 상태.

4.1 평면 분류 (3상태)

각 기하 정점의 부호 거리 `d = (p − o)·n`을 double로 계산한다. `|d| ≤ ε`이면 0, 아니면 부호대로 +1/−1이다. ε은 `ComputePieceRadius × epsilonScale(1e-5)`다. 분류는 정점 단위이므로 이웃한 면이 같은 정점을 다르게 판정하는 일이 없다. "0 정점"은 그 자체로 루프 점이 되고, 길이 0인 선분이나 끝점으로 무너지는 교점이 생기지 않는다.

4.2 걸친 면 분할과 교점 생성 (`ProcessFace`)

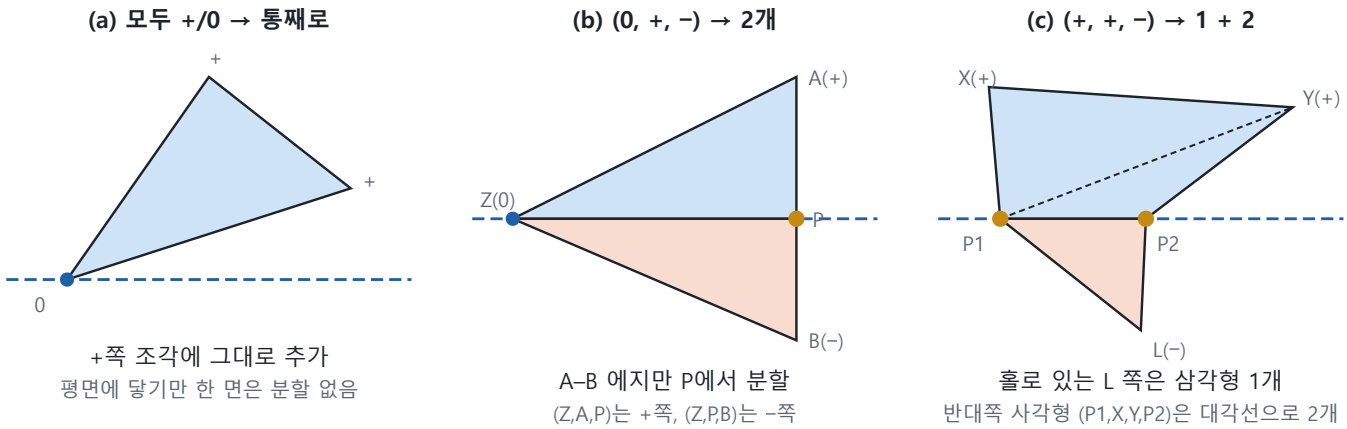


그림 6. 면 분할의 세 경우. 파란 점선이 평면, 노란 점이 새 교점. 모든 경우 원래 winding을 유지한다.

면의 부호	처리
모두 0	`CoplanarFace`로 실패(소유 규칙 미정).

−가 없음 / +가 없음	그 쪽 조각에 통째로 넣는다(`WholeCorner`는 조각별로 corner를 한 번만 복사).
Cap 면이 걸침	`CrossesCap`로 실패(cap을 다시 자르는 기능은 없음). cap 면 일부가 양쪽으로 나뉘는 경우도 같은 상태로 거절한다.
(0, +, −)	+−에지 하나만 분할 → 양쪽에 삼각형 하나씩.
(+, +, −) 류	에지 두 개 분할 → 홀로 있는 쪽 1개, 반대쪽 2개.

교점 하나는 한 번만 만든다 (`CutVertexOf`)

- key는 정렬된 기하 에지 (low, high)다. 이미 있으면 그 정점을 돌려준다. 그래서 이웃한 두 면, seam 양쪽, 두 조각이 같은 교점 번호를 공유한다.
- `t = d_low / (d_low − d_high)`, 위치 = low + t(high − low)이다. 항상 낮은 번호 정점에서 높은 번호 방향으로 계산하므로 비트 단위로 같다.
- **출처 합치기**(`MergeOrigin`): 양 끝의 origin을 (1−t), t로 섞고 같은 skinVertex를 합친다. 3개를 넘으면 `InvalidOrigin`이다. 원본 표면을 몇 번 잘라도 하위 삼각형은 원래 삼각형 하나 안에 있으므로 3개를 넘지 않는다는 것이 설계의 근거다. joint와 가중치는 보간하지 않는다.
- **corner 속성**(`CutCorner`): UV·색·법선을 렌더 에지(corner 쌍)마다 같은 t로 보간한다. 보간한 법선이 0에 가까우면 면 법선으로 대체한다.

4.3 조각 구성과 경계 계승 (`BuildPiece`)

각 조각은 쓰는 정점만 전역 번호 순서대로 압축한다. 에지 태그는 그 조각의 인접 관계에서 **다시 계산한다**. 열린 에지가 입력의 태그 에지에서 나왔으면(분할된 반쪽이면 원래 에지의) 종류를 물려받는다(`InheritedKind`). 새로 생긴 열린 에지는 평면 위에 있어야 하고 `Cut`이 된다. 입력의 cap 기록(`MeshCutRecord`)은 그 cap 면이 들어간 조각으로만 옮긴다. 결과 조각도 `ValidateMeshGeometry`를 통과해야 한다. 그래서 분할로 생긴 아주 가는 면은 `DegenerateInput`으로 거절된다.

4.4 루프 추출·검증 (`ExtractCutLoops`)

1. 양 끝이 모두 평면 위인 `Cut` 경계 에지를 면 winding 방향으로 모은다. 평면에 닿기만 하고 넘지 않는 경우는 경계가 아니므로 루프가 생기지 않는다. 접촉과 절단이 이렇게 자동으로 구분된다.
2. 정점마다 들어오는/나가는 절단 에지가 정확히 하나여야 한다. 끊긴 끝이 `Replacement/UnweldedSeam` 경계에 닿으면 `OpenLoop`, 그 밖의 경우는 `NonManifoldLoop`이다.
3. 순환을 따라 루프를 뽑고 평면 좌표계(`SlicePlaneBasis`, u×v=n)의 2D로 투영한다.
4. 루프 안 선분은 공유 정점에서만 만나야 하고, 루프끼리는 만나면 안 된다(ε보다 가까워도 실패). 위반하면 `InvalidLoop`이다.
5. 부호 면적과 포함 관계로 깊이를 정한다. 0은 외곽, 1은 구멍이다. 깊이 2 이상(구멍 속 섬)은 지원하지 않는다. winding이 깊이와 맞지 않아도 `InvalidLoop`이다.
6. `Slicer::Run`은 두 조각의 루프 크기 목록이 같은지 확인한다. 다르면 내부 오류로 `InvalidLoop`이다.

4.5 cap 삼각화 — ear clipping (`BuildCap`)

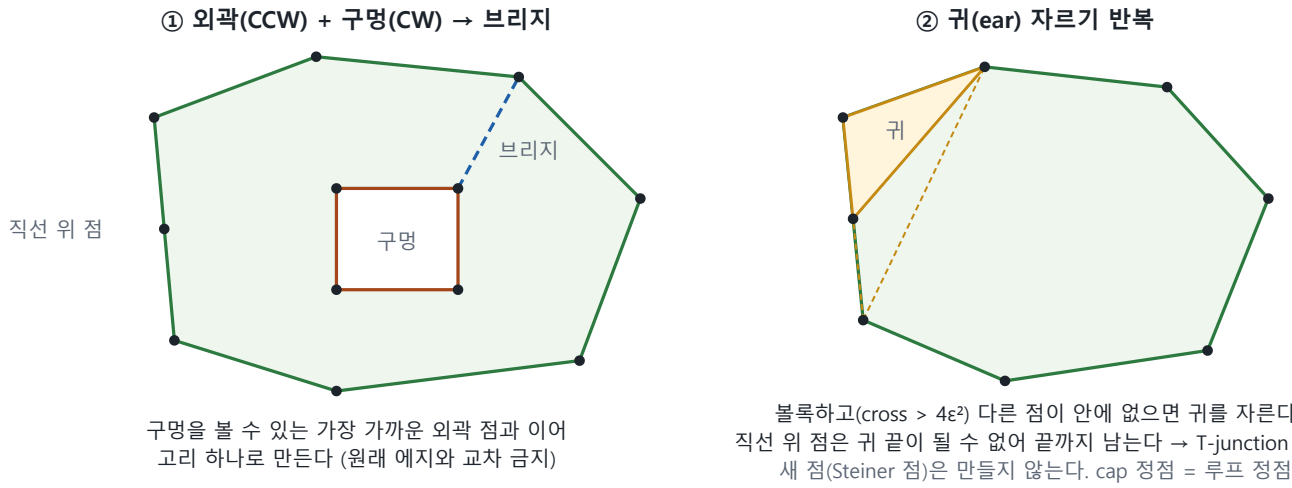


그림 7. cap 생성. 구멍은 브리지로 외곽에 잇고, 볼록한 귀를 하나씩 잘라 낸다.

- **입력**: `ExtractCutLoops`의 루프를 평면 2D 좌표의 고리로 바꾼다. 외곽은 반시계, 구멍은 시계 방향으로 맞춘다.
- **브리지**(`BridgeHole`): 구멍을 볼 수 있는 가장 가까운 정점과 잇는다. 브리지는 원래 고리의 어떤 에지와도 교차하지 않아야 하고 재료 안(외곽 안, 모든 구멍 밖)을 지나야 한다.
- **귀 자르기**(`EarClip`): `cross > 4ε²`로 엄격히 볼록하고 면적이 충분한 끝만 자른다. 남은 다른 정점이 그 삼각형 안에 있으면 건너뛴다. 귀를 찾지 못하면 `CapTriangulationFailed`다.
- **cap corner**: rim 정점마다 corner를 하나 새로 만든다(위치는 같은 기하 정점). 법선은 +쪽 조각이 −n, −쪽 조각이 +n이다. UV는 평면 좌표 × `capUvScale`이고, 색은 밝은 회색이다.
- **검증**: 모든 절단 에지는 표면과 cap에 한 번씩, 서로 반대 방향으로 쓰여야 한다. 확인되면 태그가 `Cut`에서 `CapSeam`으로 바뀐다. cap 면적 합이 루프 부호 면적 합과 상대 오차 1e-4 안에서 같아야 한다. 마지막으로 결과 전체가 `ValidateMeshGeometry`를 통과해야 한다.
- 작업은 조각의 사본(`capped`)에 하고, 모두 성공한 뒤에 바꿔 넣는다.
- **왜 틈이 없나**: cap이 표면과 같은 기하 정점을 쓰므로, 이후 `UpdatePose`가 표면을 움직이면 cap 테두리도 같이 움직인다(설계의 `FollowRim`). 큰 굽힘에서는 cap이 평면성을 잃을 수 있다. cap을 독립된 본에 붙이는 NDC 방식은 구현되지 않았다.

4.6 실패 시 원자성

층	보장
`SliceGeometry`	입력은 `const`다. 시작할 때 `out`을 비우고, 실패하면 조각·루프를 지운 채 `status`와 `detail`만 남긴다.
`BuildCap`	사본에 cap을 붙이고 모든 검사를 통과한 뒤에만 조각을 교체한다.
`SliceProceduralMesh`	`outOtherHalf == &inOutMesh`(별칭)이면 거절한다. 두 결과를 지역 `ProceduralMesh`에 `SetTopology`로 만든 뒤에만 `inOutMesh`·`outOtherHalf`·`outLoops`를 바꾼다.
`ProceduralMesh::UpdatePose`	스키닝 결과와 렌더 정점·bounds를 지역 변수에서 모두 검사한 뒤 한꺼번에 바꾼다.
`CutController::ConfirmJob`	추출·스키닝·절단·범위 승인 중 하나라도 실패하면 그 작업은 확정 결과를 갖지 않는다. 원본 숨김·분리도 하지 않는다. 오류는 `lastCutStatus`/`lastCutError`로 상태 줄에 나온다.

4.7 조각 스키닝 (매 프레임)

정점 위치 $\text{position}(v) = \Sigma_{k\le 3}\ \text{bary}_k \cdot \text{LBS}(\text{skinVertex}_k)$, $\text{LBS}(s) = \Sigma_j\ w_j \cdot M[\text{slot}][j] \cdot \text{bind}(s)$

- `M[slot][j] = meshWorld × joint.posXform(BuildSkinMatrices)`. 강체 정점(jointCount 0)은 `M[slot][0] = 메시 world(BuildRigidMatrices)`다.
- `Skinning::Apply`는 조각이 쓰는 스킨 정점만 표시해 두고, 각 LBS를 한 번만 계산해 재사용한다. 계산은 joint 오름차순, origin은 skinVertex 오름차순으로 고정한다.
- Cap 종류 정점은 위치를 유지한다. 현재 cap은 rim 정점을 공유하므로 Cap 종류 정점은 생기지 않는다.

분리된 작업의 행렬(`BuildPieceMatrices`, `CutController.cpp` 익명 namespace). `SetBreakOf`가 절단 순간의 값을 저장해 둔다: 메시 world를 뺀 slot 행렬(`breakCutMatrices`), 끊긴 노드와 그 부모의 world, 팔레트 joint가 끊긴 노드의 자손인지(`breakDetached`), +쪽이 떨어지는 쪽인지(`positiveFalls`)다. 매 프레임 조각마다 다음을 한다.

- **떨어지는 조각**: 끊기지 *않은* joint의 행렬을 `meshWorld × follow × breakCut[j]`로 바꾼다. `follow = 지금 끊긴 노드 world × inverse(절단 순간 끊긴 노드 world)`이다. 몸통 쪽 가중치가 떨어진 팔을 강체로 따라간다.
- **남는 조각**: 끊긴(자손) joint의 행렬을 같은 식으로 바꾸되, follow를 **부모** 노드로 정한다. 떨어진 joint의 기여가 절단 순간 위치에 고정된 채 몸을 따라가므로 다시 늘어나지 않는다. NDC의 "떨어진 본에 대응하는 정점 기여를 절단 당시 위치로 교체"에 해당한다. 정확한 식은 발표에서 확인하지 못했다(NDC25Reference 10절).
- 스태틱 작업과 분리 없는 작업은 `BuildJobMatrices` 결과를 그대로 쓴다.

4.8 원본 숨김

종류	방법
스태틱	`ConfirmJob` → `SetOriginalVisible(job, false)` → `ModelInstance::SetMeshVisibility(meshIndex, false)`. 그 메시 전체를 그리지 않는다. `DiscardCut`이 되돌린다.
스켈레탈	`ConfirmAll`이 모든 작업을 처리한 뒤 대상마다 `UpdateSkinMask`를 부른다. 확정된 joint 작업의 `SourceVertex::selected` 정점을 메시별 색 배열에서 R=255로 칠하고(대상·메시별 OR), `SetVertexColorOverride`로 넘긴다. 셰이더(`DefaultVS` → `DefaultPS`, `DepthOnlySkinPS`)는 보간된 `cutMask ≥ 0.5`이면 `discard`한다. 확정이 없어진 메시는 `ClearVertexColorOverride`로 되돌린다(이때 GPU idle을 기다린다).

4.9 분리 표시 (디버그 BreakConstraint)

- `SetBreakOf`: 작업 joint의 장면 노드를 끊을 노드로 정한다. 이동량은 `평면 법선 × 대상 반지름 × 0.3`이고, 부호는 그 노드의 자손 joint들이 평면 어느 쪽에 있는지(`ChildSide`)로 정한다.
- `UpdateBrokenJoints`: 끊을 노드 집합이 달라졌을 때만 `ResetBrokenConstraints` 후 다시 `BreakConstraint(offset, node)`한다.
- `ModelInstance::BreakConstraint`: 마지막 Update의 노드 world에 이동량을 더해 고정한다. 이후 `Update`는 그 노드만 부모·애니메이션을 무시하고 고정 world를 쓴다. 자손은 그 world를 부모로 삼는다. 끊긴 부분은 공중에 멈추고 몸은 계속 걷는다. **중력·충돌·물리는 없다.**
- **표시 위치**(`CutDebugView::Draw`): 끊긴 스켈레탈 작업의 두 조각은 스키닝된 자리에 그대로 그린다. 끊긴 노드가 없는 작업(스태틱 등)은 원래 자리에서 두 조각을 평면 법선 방향으로 `반지름 × 0.12`씩 벌려 그린다(+쪽은 +n, −쪽은 −n). 원본은 숨겨져 있다. 이 배치는 `4f4bfb0`에서 바뀌었고, 그 전에는 원본 오른쪽(+X)에 작업마다 한 줄로 놓았다.

5. 전체 실행 흐름과 입력 키

5.1 앱 시작과 모델 로드 (`ModelViewer::Startup`)

1. 마우스 독점 해제(커서 표시). TAA는 끄고 FXAA·HDR·SSAO는 켜며 자동 노출은 끈다. `Renderer::Initialize`, `m_ProceduralRenderer.Initialize`, IBL 텍스처를 준비한다.

2. `Renderer::LoadModel("Models/Xbot.glb")` 후 `PlayAnimation(6, true)`(걷기)와 `Resize(0.01)`를 한다. 팔레트별 joint 이름·부모를 디버그 출력에 찍는다.
3. `m_Cut.RegisterInstance(m_ModelInst, ...)`로 캐릭터를 등록한다. `AddStaticInstance`로 `cube.glb`(+X 0.9×반지름)와 `bunny.glb`(+X 1.75×반지름)를 캐릭터 반지름 0.6배 크기로 놓고 등록한다. 파일이 없으면 건너뛴다.
4. `SceneViewCamera`를 캐릭터 bounds에 맞춰 만든다.
5. 모델 로드(`ModelLoader.cpp`)는 `m_GeometryCPU`를 모든 모델에서 남겨 둔다.

5.2 한 프레임



그림 8. 한 프레임의 실행 순서. 노란 상자가 절단 시스템의 진입점과 Enter 처리다.

5.3 평면 편집 → Enter → 후속 프레임

시점	무슨 일이 일어나나
T (처음)	`ToggleEditPlane`: 편집 모드를 켜고, 첫 대상의 bounds 중심에 수평 평면을 놓는다(`MakeHorizontal`, 반 크기 = 반지름/2, 크기 제한 0.01~3×반지름). 실패했던 캐시는 다시 시도하도록 표시한다.
편집 중 매 프레임	팔레트 전체 joint로 읽은 캐시 정점을 자세가 바뀐 프레임에만 다시 스키닝한다. 평면이나 자세가 바뀐 프레임에만 `PickJoints`를 다시 돌린다. 평면 양쪽에 걸치고 절단선이 사각형과 만나는 삼각형마다 절단 joint를 하나 고른다. 기준은 가중치 > 0.05인 joint 중 **루트에 가장 가까운 것**, 같으면 가중치 합이 큰 것, 그것도 같으면 번호가 작은 것이다. joint마다 작업 key가 된다. 스태틱 메시는 바운딩 구 조기 판정 후 교차 삼각형 수로 후보가 된다. 이 단계에서는 패치를 만들지 않는다.
Enter	`RequestCut`: 후보가 하나라도 있으면 요청을 세운다. 다음 `Update`에서 이전 확정을 모두 되돌리고(`DiscardCut`), 후보마다 독립적으로 `ConfirmJob`을 한다. 각 작업은 추출 → 현재 자세 → 절단 → 범위 승인 → 저장 순서다. 그 뒤 대상마다 마스크와 끊기를 한 번에 갱신한다. 같은 프레임의 `SkinJobs`가 확정 조각을 바로 갱신한다.
이미 끊긴 joint가 있을 때 Enter	끊긴 자세에서 추출하지 않도록 그 프레임은 되돌리기만 한다. 원래 자세가 계산된 다음 프레임에 자른다. 누적 재절단이 아니라 **원본에서 다시 생성**하는 것이다.
후속 프레임	평면 편집을 끄거나 후보가 사라져도 확정 결과는 남는다. `SkinJobs`가 매 프레임 두 조각을 갱신하고 뷰가 그린다. 편집 모드를 끄면 후보 표시만 사라진다(`ApplyDiff(DiffJobs(m_Jobs, {}))`).

Ctrl+Z	`DiscardCut`: 모든 확정 결과를 지우고 원본 숨김·마스크·끊기를 되돌린다. 후보가 아닌 작업은 삭제한다.

5.4 입력 키

키	동작 (조건)
`T`	평면 편집(디버그) 모드 켜기/끄기. 모델이 있을 때만.
`Enter`	절단 요청. 편집 중이고 후보가 있을 때만.
`Ctrl`+`Z`	모든 확정 절단 되돌리기.
`V`	렌더 모드 순환: Solid → Solid+Wire → Wire → Debug(와이어 + section 분할 + 경계 튜브 + 작업 색 선). 원본과 조각에 같이 적용.
`Q` `W` `E` `R`	편집 중 기즈모 도구: 숨김 / 이동(world 축) / 회전(평면 로컬 축) / 크기. 오른쪽 버튼을 누르고 있을 때는 카메라 이동 키로 쓰인다.
왼쪽 드래그	기즈모 핸들 드래그(화살표, 두 축 사각형, 링, 크기 핸들). Alt·카메라 조작 중에는 막힌다.
오른쪽 버튼 + 마우스	둘러보기. 누른 채 `W` `A` `S` `D` 이동, `Q`/`E` 아래/위, `Shift` 빠르게, 휠 이동 속도.
가운데 드래그 / 휠	pan / 피벗 방향 줌.
`Alt`+왼쪽 / `Alt`+오른쪽 드래그	피벗 중심 궤도 회전 / 줌.
`F`	편집 중이면 평면, 아니면 모델에 카메라 맞추기.
`Space`	애니메이션 일시정지/재개.
게임패드 LB / RB	엔진의 `DebugZoom` 감소/증가.

웨이트 뷰 전용 키는 없다. 평면 아래 후보 작업이 있으면 `SetWeightView`가 그 joint를 작업 색으로 자동으로 칠한다.

6. 현재 한계와 하지 않은 것

> **사실대로 적는다.** 아래는 코드를 읽어 확인한 범위다. 실행 화면, GPU validation, 성능은 이 문서 작성 중에 확인하지 않았다.

6.1 구현되지 않은 것

- **물리 없음**: 충돌·중력·래그돌·임펄스가 없다. 스켈레탈 분리는 노드를 world에 고정하고 법선 방향으로 반지름의 0.3배 옮기는 **디버그 방식**이다. Unreal `BreakConstraint`의 물리 관절 끊기와 다르다. 스태틱 조각은 원래 자리에서 법선 방향으로 조금 벌려 그릴 뿐 떨어지거나 움직이지 않는다.
- **누적 재절단 없음**: 다음 Enter는 이전 결과를 버리고 원본에서 다시 만든다. 이미 자른 조각을 다시 자르는 흐름이 컨트롤러에 없다. 알고리즘 자체는 재절단(cutId 증가, 계승)을 지원하고 테스트하지만, cap 면을 지나는 재절단은 `CrossesCap`으로 거절한다.
- **여러 joint 병합 없음**: 걸친 joint마다 독립 작업이다. NDC의 "합친 뒤 자르기"(`RegionCutDesign.md`)는 설계안 단계로 보류 중이다.
- **메시 아일랜드, 각도 보정, 불리언 단면, 상처 데칼**: 없다.
- **cap 본 부착 없음**: cap은 rim 정점을 공유해 표면을 따라간다. 독립 강체 부착(`RigidToJoint`)은 구현하지 않았다.
- **재질·조명 없음**: 조각은 정점 색(웨이트 색·회색·cap 밝은 회색)이나 작업 색, section tint로만 그린다. 렌더러는 재질 번호를 읽지 않는다. `UpdatePose`는 법선을 갱신하지 않는다(PS가 법선을 쓰지 않음).
- **재질 section 복사 없음**: 스태틱 복사는 메시 하나를 section 하나로 만든다.
- **Enter 전 미리보기 없음**: 단면 모양은 확정 후에만 볼 수 있다(자동 미리보기는 제거했다).

6.2 알려진 제약 (코드 규칙)

- 평면과 겹치는 면 `CoplanarFace`, 구멍 속 섬(깊이 2 이상), 열린 경계에 닿는 루프 `OpenLoop`는 거절한다. 유한 사각형 밖으로 나가는 루프는 `LoopOutsidePlane`으로 확정하지 않는다.
- 절단 범위는 joint 하나의 영향 범위(가중치 > 0)다. 자손 joint에만 영향받는 삼각형은 원본에 남는다. 자손과 바깥 joint에 함께 걸친 정점은 분리 후 늘어날 수 있다(`mixedOutsideTriangles`로 센다).
- 목 joint처럼 영향이 넓은 joint는 패치가 매우 커진다(NDC25Reference 9절, CesiumMan 관찰).
- Enter 프레임에 작업마다 추출·용접·절단이 몰린다. 프레임 분산은 없다.
- 마스크는 정점 색 보간값이 0.5 이상인 픽셀만 버린다. 일부 정점만 칠해진 경계 삼각형은 부분적으로만 숨는다. 이 부분은 패치 조각이 덮는다는 가정이며, 화면으로 확인하지 않았다.
- `skinStateId`는 0만 지원한다. 분리 상태는 행렬 교체(`BuildPieceMatrices`)로 표현하고, Skinning 표는 바꾸지 않는다.

6.3 검증 상태 (문서 기록, 이번에 다시 실행하지 않음)

- `ProceduralMeshSimplification.md`는 최종 CPU 테스트 Debug/Release 각 9,470 checks 0 failed, `ProceduralMeshSmoke` 통과, ModelViewer Debug/Release 빌드 성공을 기록한다. 이 세션에서는 빌드·테스트를 실행하지 않았다.
- 같은 문서가 DirectX 실행 화면·GPU validation 확인은 하지 않았다고 기록한다. 렌더링과 조작의 실기 확인은 남아 있다.
- `tests/PlaneSlicerTests.cpp`에는 `Test`로 시작하는 함수가 81개 있다(grep 기준).

7. 파일별 역할 표

파일	주요 타입·함수	역할
`ProceduralMesh.h/.cpp`	`ProceduralMesh`, `MeshGeometry`, `MeshVertex/Corner/Face`, `MeshEdgeTag`, `MeshCutRecord`, `ProceduralMeshData`, `MeshVertexBinding`	공통 메시 객체와 자료구조 정의. Build / SetTopology / UpdatePose / SetSectionSplit / SetMesh. 검증·bounds·버전.
`BonePatch.h/.cpp`	`ExtractBonePatch`, `ReadBonePatch`, `MeshSource`, `BuildSkinMatrices`, `MakeSlotMatrices`, `ComputePoseJointWorld`, `ComputePaletteSkeleton`	스킨 원본의 joint 영향 범위 import, 스킨 행렬, 뼈대 위치·부모 계산.
`StaticMeshCopy.h/.cpp`	`CopyProceduralMeshFromStaticMesh`, `ReadStaticMesh`, `ListStaticMeshes`, `BuildRigidMatrices`	스킨 없는 메시 하나 import, 강체 world 행렬.
`Geometry/SliceTypes.h`	`SliceStatus`, `kInvalidIndex`, `V3`	공통 상태 코드와 작은 보조 함수.
`Geometry/MeshGeometry.h/.cpp`	`BuildMeshGeometry`, `ValidateMeshGeometry`, `CollectEdgeUsage`, `TotalArea`, `SlicePlane`, `MakePlaneBasis`	용접, 입력 검증(설계 3.1), 에지 사용, 면적·반지름.
`Geometry/Slice.h/.cpp`	`SliceProceduralMesh`, `SliceGeometry`, `Slicer`(익명), `SliceCapOption`, `SliceOptions`	공개 절단 API와 분류·면 분할·조각 구성·경계 계승.
`Geometry/SliceCap.cpp`	`ExtractCutLoops`, `BuildCap`, `BridgeHole`, `EarClip`	루프 추출·2D 검증·포함 깊이, ear clipping cap.

`Geometry/Skinning.h/.cpp`	`Skinning`, `SkinVertex`, `Apply`, `SkinPoint`, `RigidPoint`, `InJointSet`	공유 스킨 표와 위치 계산, joint 집합 판정.
`Geometry/CutPlane.h/.cpp`, `CutPlaneGizmo.cpp`	`CutPlane`, `ContainsLoops`, `PickSegments`, `MakeHorizontal`, 기즈모 함수들	world 유한 사각 평면, 범위 승인, 뼈 선분 표시, 마우스 기즈모(피킹·드래그·굽기).
`Geometry/CutPick.h/.cpp`	`CutPick::PickJoints`, `CountCrossedTriangles`, `SphereMayCross`	교차 삼각형 → 절단 joint 선택, 스태틱 교차 수, 바운딩 구 조기 판정.
`Runtime/CutJob.h`	`CutJob`, `CutJobKey`, `CutTarget`, `CutTargetSource`, `CutStaticMesh`, `DiffJobs`, `GetJobColor`	작업 식별·상태·목록 비교. Model/Core에 의존하지 않음.
`Runtime/CutController.h/.cpp`	`CutController`: Update, ConfirmAll/ConfirmJob, ExtractJob, SkinPatch, SliceWithPlane, SkinJobs, SetBreakOf, UpdateSkinMask, UpdateBrokenJoints, DiscardCut	입력 요청·대상·후보·확정·복원 조율과 계측.
`Rendering/RenderBake.h/.cpp`	`BuildRenderMesh`, `UpdateRenderPositions`, `BakeBoundaryDebug`, `BakeLoopDebug`	기하 → 렌더 캐시 굽기(section 규칙), 디버그 튜브. D3D12에 의존하지 않음.
`Rendering/ProceduralMeshRenderer.h/.cpp`	`ProceduralMeshRenderer`: Draw, NotifySubmitted, ReleaseMesh, Shutdown	D3D12 업로드·그리기, 버전 비교, fence 기반 버퍼 재사용, PSO 5개.
`Rendering/ProceduralMeshVS/PS.hlsl`	Transform(b0), SectionTint(b1)	World×ViewProj 변환, 정점 색/section 색/와이어 색.
`Debug/CutDebugView.h/.cpp`	`CutDebugView`: Sync, Draw, DrawUI, Release, `RenderMode`	확정 조각 배치·표시, 경계 튜브, 평면·뼈 선분·기즈모, 상태·계측 줄.
`Debug/SceneViewCamera.h/.cpp`	`SceneViewCamera`	Unity Scene 뷰식 카메라 조작.
`tests/PlaneSlicerTests.cpp`, `ProceduralMeshSmoke.cpp`	독립 실행 테스트	Geometry·RenderBake·ProceduralMesh를 GPU 없이 검사. 빌드 명령은 파일 머리에 있다.
`ModelViewer/ModelViewer.cpp`	Startup, AddStaticInstance, Update, RenderScene, RenderProceduralMeshes, SetWeightView, RenderUI	앱. 모델 로드·등록, 키 입력, 프레임 순서, 그리기 호출.
`Model/Model.h/.cpp`	`ModelInstance`: SetMeshVisibility, SetVertexColorOverride, ClearVertexColorOverride, BreakConstraint, ResetBrokenConstraints, SetPosition	원본 쪽 확장: 메시 숨김, 정점 색 마스크, 노드 고정, 위치 지정.
`Model/ModelLoader.cpp`	`m_GeometryCPU` 보존	모든 모델의 CPU 정점 사본을 남긴다(절단 읽기용).
`Model/Shaders/DefaultVS·PS`, `DepthOnlySkinPS`	`cutMask`	정점 색 R을 넘겨 0.5 이상이면 discard.

8. 용어집

용어	뜻
joint	transform 하나(이름, 부모, 로컬 변환). NDC/Unreal의 Bone. 웨이트가 붙는 대상이다.
segment	부모 joint → 자식 joint 선분. 표시용이며 데이터가 아니다(`PickSegments`).
팔레트 (palette)	MiniEngine의 스킨 joint 묶음(`SkinPalette`). 메시는 팔레트 하나를 쓴다.
패치 / 복사 범위	선택 joint에 영향받는 삼각형 집합을 ProceduralMesh로 복사한 것.
작업 (CutJob)	후보 하나(대상 × joint 또는 대상 × 스태틱 메시)의 절단 상태와 결과.
후보 (candidate)	이번 프레임 평면이 지나는 작업. Enter 때 잘린다.
확정 (confirmed)	절단과 범위 승인에 성공해 두 조각을 가진 작업.
기하 정점 (MeshVertex)	위치를 소유하는 점. 용접 결과다.
corner (MeshCorner)	렌더 정점. 기하 정점 번호 + 법선·UV·색. UV seam·hard normal이 여기서 갈린다.
origin / bary	기하 정점이 Skinning 표의 원본 정점 최대 3개를 어떤 비율로 섞는지.
용접 (weld)	같은 슬롯·바인드 위치·스킨의 렌더 정점을 한 기하 정점으로 합치는 것.
Replacement	복사 범위의 열린 경계 에지(원본과 맞닿는 곳).
UnweldedSeam	위치가 같지만 스킨이나 슬롯이 달라 합치지 않은 seam 경계.
Cut / CapSeam	이번 절단이 만든 열린 경계 / cap으로 닫힌 뒤의 내부 에지.
루프 (loop)	평면 위 절단 경계 에지가 이루는 닫힌 고리. 외곽(깊이 0)과 구멍(깊이 1).
cap	루프를 채우는 단면 삼각형들(`MeshFaceKind::Cap`).
ear clipping	다각형에서 볼록한 꼭짓점 삼각형(귀)을 하나씩 잘라 내는 삼각화.
브리지 (bridge)	구멍과 외곽을 잇는 에지. 구멍 있는 다각형을 고리 하나로 만든다.
cutId	절단 번호. 메시 안 가장 큰 값 + 1. 원본은 0.
ε (epsilon)	조각 반지름 × 1e-5. 평면 위 판정, 퇴화 면 기준(면적 ≤ ε^2), cap 귀 기준.
LBS	Linear Blend Skinning. Σ weight × joint 행렬 × 바인드 위치.
강체 정점	jointCount 0인 스킨 정점(스태틱 메시). 메시 world 하나로 옮긴다.
BreakConstraint (여기서)	노드 world를 고정하고 이동량을 더하는 디버그 분리. 물리 아님.
마스크	정점 색 R로 원본의 복사 범위를 숨기는 것(`SetVertexColorOverride`).
버전 (vertex/topology)	메시 내용이 바뀔 때마다 새로 받는 전역 유일 번호. 렌더러 업로드 판단용.
fence	GPU 명령 완료 표지. 버퍼를 언제 재사용해도 되는지 판단한다.
원자성	실패하면 입력·출력이 하나도 바뀌지 않는다는 계약.

부록 A. 문서–코드 차이

코드를 기준으로 판단했다. "이력 문서"로 표시된 문서(PlaneSlicingDesign, UnrealAlignmentDesign 등)는 과거 이름을 일부러 남겼을 수 있으므로 심각도를 낮게 적었다. 아무것도 고치지 않았다.

#	문서 / 위치	문서 내용	코드 (기준)
1	`NDC25Reference.md` 2절 용어표	ProceduralMesh ↔ "`ProceduralMesh` + `PatchPiece`". BreakConstraint "(미구현)", 버텍스 컬러 마스킹 "(미구현) 원본 숨김"	`PatchPiece`는 없다. 마스킹(`UpdateSkinMask`)과 디버그 BreakConstraint(`UpdateBrokenJoints`)는 구현되어 있다. 같은 문서 8절과도 서로 맞지 않는다.
2	`NDC25Reference.md` 5절 예외 항목	`BonePatchSource::mixedOutsideTriangles`	타입 이름이 `MeshSource::mixedOutsideTriangles`로 바뀌었다.
3	`NDC25Reference.md` 5절 채택 해석 1	절단 위치의 segment를 찾고 그 **부모 joint**의 영향 범위를 복사	현재는 `CutPick::PickJoints`가 교차 삼각형마다 **루트에 가장 가까운** 후보 joint(가중치 > 0.05)를 고른다. segment는 표시용이다(JointCutDesign으로 대체). 5절에는 이 변경이 적혀 있지 않다.
4	`CodeMap.md` 1절 "확정" 행	`ConfirmJob → UpdateSkinMask, UpdateBrokenJoints`	`ConfirmJob`은 `SetOriginalVisible`·`SetBreakOf`만 부른다. `UpdateSkinMask`·`UpdateBrokenJoints`는 `ConfirmAll`이 모든 작업 뒤에 대상마다 한 번 부른다.
5	`Runtime/CutController.h` 주석	`Update`: "메시 교차 -> 작업 목록 -> 스키닝 -> Enter 요청 시 확정". 파일 머리 "선분별 절단 작업, 설계: PerSegmentCutDesign"	실제 순서는 교차 → 작업 목록 → **확정** → **스키닝**(`ConfirmAll`이 `SkinJobs`보다 먼저)이다. 작업 단위도 선분이 아니라 joint/스태틱 메시다.
6	`Debug/CutDebugView.h` 머리 주석	"컨트롤러를 읽기만 한다"	`Sync`가 공유 받은 확정 메시에 `SetSectionSplit`을 부른다. 렌더 캐시를 다시 굽고 버전을 바꾼다. 기하는 바꾸지 않는다.
7	`ModelViewer.cpp` 정적 모델 상수 주석 (`223d2a1`)	"조각은 원본에 붙으므로 옆 공간이 필요 없다"	`223d2a1` 시점의 `CutDebugView::Draw`는 끊긴 노드가 없는 작업의 조각을 +X로 옮긴 줄에 놓아 주석과 달랐다. **`4f4bfb0`에서 원래 자리 + 법선 방향 벌림으로 바뀌어 지금은 주석과 맞는다.** 화면으로는 확인하지 않았다.
8	`UnrealAlignmentDesign.md` 4·5·5.1절 이력	`PatchPiece& inOutMesh`, `MakeWeldInput → BuildPatchPiece`, `BuildRigidMatrices + Skinning::Apply`, "`job.patch`는 면적 비교에 남기므로 사본을 자른다"	`ProceduralMesh& inOutMesh`, `ProceduralMesh::Build`, `UpdatePose`다. `ConfirmJob`은 자르기 전에 면적을 계산하고 같은 지역 메시를 제자리에서 자른다. 문서 머리에 "과거 설계 이력"이라고 밝혀 두었다.

9	`PlaneSlicingDesign.md` 이력	`PatchNode/PatchPiece`, `WeldInput`, `SkinPatchPiece`, `PatchPiece::originNodes`, `UpdateRenderPositions` → `UpdateVertices`, 키(T 슬라이스, [ ], G, M, L, Z)	이름은 Mesh* 계열과 `Skinning::Apply/UpdatePose`로 바뀌었다. 원본 스킨은 공유 `Skinning` 표에 있다. 키는 5.4절과 다르다. 알고리즘(3상태 분류, 에지 캐시, 계승, 루프 검증, ear clipping)은 코드와 일치한다.
10	`UnrealAlignmentDesign.md` U-4 행	스켈레탈 마스크: color/depth/**shadow**	코드에서 확인한 discard는 `DefaultPS`와 `DepthOnlySkinPS`다. 그림자 패스가 어느 PS를 쓰는지는 이번에 추적하지 않았다(미확인).
11	추출 기준 (문서 다수)	"minWeight를 넘는 가중치"(일반 서술)	후보 선택은 `kPickMinWeight = 0.05`, Enter 추출·마스크는 `minWeight = 0.0`(가중치 > 0)이다. 기준이 다르다는 점이 한곳에 정리되어 있지 않다.
12	`BonePatch.h` `SourceVertex` 주석	"`ProceduralMeshData::vertices`와 같은 순서"	import 데이터 기준으로는 맞다. 용접 뒤에는 기하 정점 순서와 다르다(corner 순서와는 절단 전까지 같다). 오해 소지만 있고 버그는 아니다.

일치를 확인한 주요 서술: CodeMap 2·3절의 데이터 주인·파일 역할, ProceduralMeshSimplification의 "없앤 것"과 보존한 계약(별칭 출력 거절, const Skinning 공유, shared_ptr 수명, 다음 Enter 재생성), NDC25Reference 8절 대응표, PlaneSlicingDesign 11절 구현 노트(ear clipping 직접 구현, 출력 검증, NoCap에서도 루프 추출).

부록 B. 확인하지 못한 것

- **실행 화면**: 앱을 빌드·실행하지 않았다(다른 세션이 코드를 수정 중이라 이번 작업 범위에서 제외). 조각 위치, 마스크 결과, 기즈모 동작, 분리 모습은 코드 읽기로만 설명했다.
- **테스트 수와 통과 여부**: 9,470 checks는 문서 기록이며 다시 실행하지 않았다.
- **그림자 패스의 마스크 처리**(부록 A #10).
- **Unreal 내부 구현**: 섹션 bounds 조기 판정, 로컬 공간 평면 변환은 문서에서도 추론이다.
- **NDC 원본의 세부**: 기여 교체의 정확한 식, cap 본 부착 방식, Bone 선택 방법(NDC25Reference 10절).
- **성능 수치**: 계측 코드(`CutFrameStats`, `PhaseTimer`)는 있지만 값은 측정하지 않았다.
- **이전 코드**: 통합 전 흐름(그림 3 왼쪽)은 `cb1c12a^` 헤더의 이름과 문서로 그렸다. 이전 구현 본문은 읽지 않았다.

작성: 2026-10-22. 기준 커밋 `223d2a1`, 표시 배치는 `4f4bfb0` 반영. 저장소의 코드·Docs·git 상태는 수정하지 않았다.